Programme Name and Semester: BCA and 4th Sem
Course Name (Course Code): Object Oriented Programming in JAVA Lab (BCA49108)
Academic Session: 2025-2026

Laboratory Manual

(Object Oriented Programming in JAVA Lab and BCA49108)

Table of Contents

	5	Write a Java program to create a class `Rectangle` with private data members for length and breadth. Calculate and display the area using public methods.	
	6	Write a Java program to demonstrate the use of **multiple objects of a class** and show how each object maintains its own copy of data members.	
	7	Write a Java program to demonstrate the difference between **instance variables and local variables** in a	
	8	Write a Java program to create two classes in the same package and demonstrate **member accessibility** using access control modifiers.	
Module3	1	Design a Java program to implement an **Employee Management System** using inheritance. • Create a base class Employee with data members: empId, name, and basicSalary. • Create derived classes PermanentEmployee and ContractEmployee. • Use **constructors and constructor overloading** to initialize employee details. • Override a method calculateSalary() in derived classes to compute salary differently. • Demonstrate **runtime polymorphism** using base class references.	
	2	Create a Java program to calculate the area of different geometric shapes. • Create a class Shape with **overloaded methods** named area() for circle, rectangle, and triangle. • Use a **static data member** to count how many times area calculation is performed. • Use a static method to display the total count. • Accept values using **command line arguments**.	
	3	Develop a Java program to manage student examination results. • Create a base class Student with roll number and name. • Create a derived class Result containing marks of multiple subjects stored in an **array**. • Calculate total marks, percentage, and grade. • Use **string operations** to format and display the result sheet.	

		• Override a method displayResult() in the derived class.	
	4	Design a Java program for a simple banking system. • Create an interface Account with methods deposit() and withdraw(). • Implement the interface in classes SavingsAccount and CurrentAccount. • Use **constructors** to initialize account details. • Demonstrate **polymorphism** using interface references. • Validate withdrawal conditions.	
	5	Create a Java application using **user-defined packages**. • Create a package utility containing a class MathUtility with **static methods** for basic operations. • Create another class in a different package that imports and uses these methods. • Demonstrate the use of **access control modifiers**. • Accept inputs through **command line arguments**.	
Module4	1	Design a Java program to accept marks of a student in multiple subjects. • If marks are less than 0 or greater than 100, throw a **user-defined exception** InvalidMarksException. • Use try, catch, and throw to handle invalid input. • Display total marks and percentage if input is valid. • Ensure proper exception messages are shown.	
	2	Develop a Java program for a simple banking system. • Create methods for deposit() and withdraw(). • Use throws to propagate exceptions from methods. • Handle conditions like insufficient balance using exceptions. • Demonstrate multiple catch blocks.	
	3	Write a Java program to create multiple threads by extending the Thread class. • Each thread should display a task name and sleep for a fixed interval.	

		• Demonstrate thread creation, execution, and termination. • Show the effect of thread priority.	
	4	Create a Java program where multiple threads perform arithmetic operations. • Handle runtime exceptions such as division by zero using try–catch. • Each thread should handle its own exception. • Display thread name along with exception message. • Ensure the program continues execution even after exceptions.	

Module5	1	Design a Java program to manage student records using ArrayList.
		• Create a Student class with roll number, name, and marks.
		• Store multiple student objects in an ArrayList.
		• Perform the following operations:
		  ○ Add a new student
		  ○ Display all students
		  ○ Search a student by roll number
		  ○ Remove a student
		• Demonstrate iteration using Iterator and for-each loop.	

	2	Create a Java program to maintain an employee directory using Vector.
		• Store employee ID and name in a Vector.
		• Demonstrate:
		  ○ Adding elements
		  ○ Updating an element
		  ○ Removing an element
		  ○ Traversing the Vector using Enumeration
		• Explain thread-safety of Vector using comments.	

	3	Develop a Java program to manage book information using HashSet. • Create a Book class with bookId, title, and author. • Override equals() and hashCode() methods. • Store multiple Book objects in a HashSet.	

		• Ensure duplicate books are not stored. • Display all unique books.	

	4	Create a Java application that uses **multiple collection classes** together.
		• Use:
		  ○ ArrayList to store student names
		  ○ HashSet to store unique course names
		  ○ HashMap to map student names with courses
		• Perform add, update, delete, and display operations.
		• Demonstrate the use of List, Set, and Map interfaces.	

Aim / Purpose of the Experiment

Assignment 1

The aim of this assignment is to introduce students to **basic Java programming concepts**, including user input, variable declaration, arithmetic operations, and formatted output. This assignment helps students understand the structure of a Java program and the process of compiling and executing simple Java applications.

Assignment 2

The aim of this assignment is to provide hands-on practice in Java programming by using **user input, variable manipulation, conditional statements, and looping constructs**. It reinforces logical thinking and the application of arithmetic and decision-making operations in Java programs.

Assignment 3

The aim of this assignment is to enable students to **apply object-oriented programming principles in Java** by designing and implementing a simple **Library Management System**. The objectives include understanding class design, encapsulation, object interaction, handling basic operations, and developing problem-solving skills using OOP concepts.

Assignment 4

The aim of this assignment is to assess the student's ability to **design and implement Java classes** using object-oriented principles. It focuses on creating classes with instance variables, constructors, and methods, along with the use of conditional statements, input validation, and user interaction in Java.

Assignment 5

The aim of this assignment is to develop an understanding of **inheritance and class hierarchy** in Java. Students are required to model a real-world staff database by deriving specialized classes such as Teacher, Officer, and Typist from a base class. This assignment emphasizes code reusability, hierarchical relationships, and testing class functionality through a driver program.